



هدف پروژه:

ایجاد یک API ساده برای مدیریت وظایف (Tasks) با استفاده از FastAPI و PostgreSQL، که قابلیت‌های CRUD (ایجاد، خواندن، به‌روزرسانی، و حذف) را فراهم کند. این پروژه باید به طور کامل روی یک سرور لینوکسی دیپلوی شده و مستندات مناسب داشته باشد.

جزئیات پروژه

۱. عملکرد (API Endpoints):

- GET /tasks: دریافت لیست تمامی وظایف.
- POST /tasks: ایجاد یک وظیفه جدید.
- GET /tasks/{task_id}: دریافت جزئیات یک وظیفه مشخص.
- PUT /tasks/{task_id}: به‌روزرسانی یک وظیفه.
- DELETE /tasks/{task_id}: حذف یک وظیفه.

۲. مدل داده‌ها (Database Schema):

مدل برای ذخیره اطلاعات وظایف در دیتابیس PostgreSQL به شکل زیر تعریف شود:

• Task:

• id (عدد صحیح، کلید اصلی)

• title (رشته، اجباری)

• description (رشته، اختیاری)

• is_completed (بولین، پیش‌فرض: False)

• created_at (زمان ایجاد، به طور خودکار مقداردهی شود)



۳. تکنولوژی‌های مورد نیاز:

1. زبان برنامه‌نویسی: Python

2. فریم‌ورک: FastAPI

3. پایگاه داده: PostgreSQL

4. وابستگی‌ها: مدیریت با requirements.txt

5. مستندات: Swagger (خروجی پیش‌فرض FastAPI در مسیر docs/).

6. دیپلوی روی سرور لینوکس:

• استفاده از gunicorn و uvicorn.

• پیکربندی سرویس با systemd.

۴. ساختار پروژه:

└── /app

└── main.py # فایل اصلی اپلیکیشن

└── models.py # تعریف مدل‌های دیتابیس

└── schemas.py # تعریف اسکیمای Pydantic

└── crud.py # توابع CRUD

└── database.py # مدیریت اتصال به دیتابیس

└── /routers

└── tasks.py # روتر مربوط به وظایف

└── config.py # تنظیمات کلی

└── requirements.txt # وابستگی‌ها

└── README.md # مستندات



۵. وظایف نیروی استخدامی:

1. ایجاد پروژه:

- طراحی و پیاده‌سازی API با استفاده از FastAPI و PostgreSQL.

- مدل‌سازی دیتابیس و اجرای عملیات CRUD.

2. مستندات API:

- اطمینان از در دسترس بودن Swagger در مسیر /docs.

3. ورژن کنترل:

- ذخیره پروژه در یک مخزن GitHub یا GitLab و اشتراک‌گذاری لینک.

4. دیپلوی روی سرور:

- اجرای پروژه روی یک سرور لینوکسی.

- تنظیم gunicorn و systemd برای اجرای سرویس.

5. ارائه خروجی:

- ارائه مستندات README.md شامل:

- مراحل نصب و اجرای پروژه.

- توضیح مسیرهای (Endpoints) API.

- دستورالعمل‌های دیپلوی.



۶. نکات ارزیابی:

1. درک و اجرای دقیق الزامات پروژه.

2. ساختار کد تمیز و مدولار.

3. پیکر بندی صحیح دیتابیس و FastAPI.

4. مستند سازی مناسب و قابل فهم.

5. اجرای موفق روی سرور لینوکسی.